1. Introducción y alcance

Se propone una base de datos relacional para una red social de estudiantes que permita crear perfiles, registrar intereses, seguir compañeros, publicar contenidos, administrar grupos y organizar eventos. Como no se suministró el modelo lógico de la actividad anterior, se construyó un modelo lógico de referencia a partir de los requisitos del enunciado; debe contrastarse con el modelo previo antes de la entrega definitiva.

2. Selección del SGBD

Se elige PostgreSQL por su soporte sólido de integridad referencial, transacciones ACID, restricciones CHECK, índices, tipos TIMESTAMPTZ, licencia abierta y herramientas como pgAdmin 4. Es adecuado para relaciones muchos-a-muchos y crecimiento funcional. Desventajas: administración inicial más técnica y necesidad de ajustar rendimiento conforme aumenta el volumen.

MySQL: ventajas: amplia adopción, documentación y facilidad de alojamiento; desventajas: diferencias de sintaxis y características avanzadas según versión/motor. SQLite: ventajas: instalación mínima y uso local sencillo; desventajas: menos conveniente para múltiples usuarios concurrentes y administración centralizada de una red social. PostgreSQL ofrece un equilibrio favorable entre integridad, concurrencia y extensibilidad.

3. Transformación del modelo lógico al físico

Cada entidad se convirtió en una tabla; sus atributos se asignaron a columnas SQL. Las relaciones 1:N se implementaron mediante claves foráneas en la tabla hija. Las relaciones N:M se materializaron como tablas puente con claves primarias compuestas: estudiante_intereses, membresias_grupo y asistencias_evento. La relación de seguimiento se representa con conexiones, que referencia dos veces a estudiantes. Se usa snake_case, nombres descriptivos en plural, BIGINT para identificadores y TIMESTAMPTZ para fechas con zona horaria.

4. Inventario detallado de tablas y atributos

estudiantes

Campo	Tipo SQL	Restricción
id_estudiante	BIGINT GENERATED ALWAYS AS IDENTITY	PK
nombre	VARCHAR(100)	NOT NULL
apellido	VARCHAR(100)	NOT NULL
correo	VARCHAR(254)	NOT NULL UNIQUE
programa	VARCHAR(120)	NOT NULL
biografia	TEXT	Opcional
fecha_registro	TIMESTAMPTZ	NOT NULL DEFAULT CURRENT_TIMESTAMP

PK: id_estudiante. Identificador único estable e independiente de atributos que pueden cambiar.

intereses

Campo	Tipo SQL	Restricción
id_interes	BIGINT GENERATED ALWAYS AS IDENTITY	PK

Campo	Tipo SQL	Restricción
nombre	VARCHAR(80)	NOT NULL UNIQUE

PK: id_interes. Identificador único estable e independiente de atributos que pueden cambiar.

estudiante_intereses

Campo	Tipo SQL	Restricción
id_estudiante	BIGINT	PK, FK estudiantes
id_interes	BIGINT	PK, FK intereses

PK: id_estudiante, id_interes. Clave compuesta para impedir duplicados de la misma asociación.

conexiones

Campo	Tipo SQL	Restricción
id_seguidor	BIGINT	PK, FK estudiantes
id_seguido	BIGINT	PK, FK estudiantes
fecha_conexion	TIMESTAMPTZ	NOT NULL DEFAULT CURRENT_TIMESTAMP

PK: id_seguidor, id_seguido. Clave compuesta para impedir duplicados de la misma asociación.

publicaciones

Campo	Tipo SQL	Restricción
id_publicacion	BIGINT GENERATED ALWAYS AS IDENTITY	PK
id_autor	BIGINT	NOT NULL FK estudiantes
contenido	TEXT	NOT NULL
fecha_publicacion	TIMESTAMPTZ	NOT NULL DEFAULT CURRENT_TIMESTAMP

PK: id_publicacion. Identificador único estable e independiente de atributos que pueden cambiar.

grupos

Campo	Tipo SQL	Restricción
id_grupo	BIGINT GENERATED ALWAYS AS IDENTITY	PK
id_creador	BIGINT	NOT NULL FK estudiantes
nombre	VARCHAR(120)	NOT NULL
descripcion	TEXT	Opcional
fecha_creacion	TIMESTAMPTZ	NOT NULL DEFAULT CURRENT_TIMESTAMP

PK: id_grupo. Identificador único estable e independiente de atributos que pueden cambiar.

membresias_grupo

Campo	Tipo SQL	Restricción
id_grupo	BIGINT	PK, FK grupos
id_estudiante	BIGINT	PK, FK estudiantes
fecha_union	TIMESTAMPTZ	NOT NULL DEFAULT CURRENT_TIMESTAMP

PK: id_grupo, id_estudiante. Clave compuesta para impedir duplicados de la misma asociación.

eventos

Campo	Tipo SQL	Restricción
id_evento	BIGINT GENERATED ALWAYS AS IDENTITY	PK
id_organizador	BIGINT	NOT NULL FK estudiantes
id_grupo	BIGINT	NULL FK grupos
titulo	VARCHAR(150)	NOT NULL
descripcion	TEXT	Opcional
lugar	VARCHAR(180)	Opcional
inicio	TIMESTAMPTZ	NOT NULL
fin	TIMESTAMPTZ	NOT NULL

PK: id_evento. Identificador único estable e independiente de atributos que pueden cambiar.

asistencias_evento

Campo	Tipo SQL	Restricción
id_evento	BIGINT	PK, FK eventos
id_estudiante	BIGINT	PK, FK estudiantes
estado	VARCHAR(15)	NOT NULL DEFAULT 'confirmado'

PK: id_evento, id_estudiante. Clave compuesta para impedir duplicados de la misma asociación.

5. Justificación de tipos y tamaños

BIGINT admite identificadores de gran capacidad y evita migraciones tempranas por agotamiento de INT; supone mayor almacenamiento que INT, aceptable para esta propuesta. VARCHAR(254) para correo y VARCHAR(80–180) para nombres/títulos/lugar establecen límites de negocio y validación; TEXT se reserva a biografías, descripciones y publicaciones sin longitud máxima predefinida. TIMESTAMPTZ permite conservar instantes correctamente al operar entre zonas horarias. Las tablas puente reutilizan BIGINT para coincidir con las PK referenciadas.

6. Relaciones y acciones referenciales

estudiantes.id_estudiante (padre) → estudiante_intereses.id_estudiante (hija). ON DELETE CASCADE: elimina registros dependientes al eliminar el padre. ON UPDATE CASCADE: propaga un eventual cambio de identificador.

intereses.id_interes (padre) → estudiante_intereses.id_interes (hija). ON DELETE CASCADE: elimina registros dependientes al eliminar el padre. ON UPDATE CASCADE: propaga un eventual cambio de identificador.

estudiantes.id_estudiante (padre) → conexiones.id_seguidor (hija). ON DELETE CASCADE: elimina registros dependientes al eliminar el padre. ON UPDATE CASCADE: propaga un eventual cambio de identificador.

estudiantes.id_estudiante (padre) → conexiones.id_seguido (hija). ON DELETE CASCADE: elimina registros dependientes al eliminar el padre. ON UPDATE CASCADE: propaga un eventual cambio de identificador.

estudiantes.id_estudiante (padre) → publicaciones.id_autor (hija). ON DELETE CASCADE: elimina registros dependientes al eliminar el padre. ON UPDATE CASCADE: propaga un eventual cambio de identificador.

estudiantes.id_estudiante (padre) → grupos.id_creador (hija). ON DELETE RESTRICT: impide borrar el registro padre mientras haya contenido que deba conservarse. ON UPDATE CASCADE: propaga un eventual cambio de identificador.

grupos.id_grupo (padre) → membresias_grupo.id_grupo (hija). ON DELETE CASCADE: elimina registros dependientes al eliminar el padre. ON UPDATE CASCADE: propaga un eventual cambio de identificador.

estudiantes.id_estudiante (padre) → membresias_grupo.id_estudiante (hija). ON DELETE CASCADE: elimina registros dependientes al eliminar el padre. ON UPDATE CASCADE: propaga un eventual cambio de identificador.

estudiantes.id_estudiante (padre) → eventos.id_organizador (hija). ON DELETE RESTRICT: impide borrar el registro padre mientras haya contenido que deba conservarse. ON UPDATE CASCADE: propaga un eventual cambio de identificador.

grupos.id_grupo (padre) → eventos.id_grupo (hija). ON DELETE SET NULL: mantiene el evento y desvincula el grupo eliminado. ON UPDATE CASCADE: propaga un eventual cambio de identificador.

eventos.id_evento (padre) → asistencias_evento.id_evento (hija). ON DELETE CASCADE: elimina registros dependientes al eliminar el padre. ON UPDATE CASCADE: propaga un eventual cambio de identificador.

estudiantes.id_estudiante (padre) → asistencias_evento.id_estudiante (hija). ON DELETE CASCADE: elimina registros dependientes al eliminar el padre. ON UPDATE CASCADE: propaga un eventual cambio de identificador.

7. Estructura del script DDL y ejecución

El archivo modelo_fisico.sql inicia una transacción, crea el esquema red_pascualina, define nueve tablas con CREATE TABLE, añade las claves foráneas mediante ALTER TABLE, crea índices de consulta y confirma con COMMIT. Las restricciones CHECK evitan seguirse a sí mismo, fechas de evento inválidas, publicaciones vacías y estados de asistencia fuera del catálogo.

Procedimiento en pgAdmin 4: (1) instalar PostgreSQL y pgAdmin; (2) crear una base de datos vacía, por ejemplo pascualina; (3) abrir Query Tool; (4) cargar modelo_fisico.sql; (5) ejecutar; (6) refrescar Schemas > red_pascualina > Tables; (7) verificar las nueve tablas y sus constraints. Para evidencia de ejecución, agregar capturas propias de pgAdmin al informe. No se afirma aquí que el script se haya ejecutado en una instancia real.

8. Pruebas de validación sugeridas

Comprobar que la consulta de information_schema incluida al final del SQL devuelve nueve tablas. Insertar dos estudiantes, un interés, un grupo y un evento válidos; comprobar que las claves foráneas impiden insertar referencias inexistentes. Intentar seguirse a sí mismo y crear un evento con fin anterior al inicio: ambos deben fallar. Verificar CASCADE en membresías y RESTRICT al borrar un organizador de evento.

9. Reflexión individual

La principal dificultad conceptual fue distinguir relaciones uno-a-muchos de muchos-a-muchos y decidir cuándo conviene impedir un borrado o propagarlo. El diseño me permitió comprender que las claves primarias no solo identifican filas, sino que sustentan la consistencia de las relaciones. También observé que escoger tipos de datos implica equilibrar almacenamiento, límites de negocio y crecimiento. Como mejora futura incorporaría moderación, roles, auditoría, privacidad y pruebas automatizadas. Esta reflexión debe adaptarse a lo que efectivamente realicé y comprobé.

10. Conclusión

El modelo físico propuesto cubre los requisitos funcionales básicos y define restricciones de integridad verificables. Su utilidad depende de contrastarlo con el modelo lógico previo, ejecutarlo en PostgreSQL y documentar evidencias reales de funcionamiento.

Referencias

Enunciado de Tarea 3: Red Social Estudiantil Pascualina, material proporcionado por el curso.

Canepa, G. (2016). PostgreSQL Database Handbook.
http://dev.cs.ovgu.de/db/PostgreSQL-Database-Handbook.pdf

Elmasri, R. y Navathe, S. A. (2007). Fundamentos de sistemas de bases de datos (5.ª ed.). Pearson.

PostgreSQL Global Development Group. Documentación oficial: https://www.postgresql.org/docs/